

Dependency-Induced Waste in Continuous Integration: A Large-Scale Study of the JavaScript Ecosystem

Rintaro Kato

TABLE I
PROJECT SELECTION PIPELINE.

Step	Criterion	#Projects
PS1	JavaScript, ≥ 10 stars (SEART)	261,739
PS2	≥ 10 commits during 2024-03-01 – 2026-03-01	100,811
PS3	GitHub Actions adopted (<code>.github/workflows/*.yaml</code>)	16,226
PS4	≥ 10 GitHub Actions executions	(TBD)
PS5	npm adopted (<code>package.json</code> exists)	13,991

Abstract—

I. INTRODUCTION

II. RELATED WORK

III. METHODOLOGY

Figure ?? shows an overview of our study pipeline, which consists of three stages: (1) Project Selection (PS), (2) Data Curation (DC), and (3) Skip Analysis.

A. Project Selection

We collected JavaScript projects from GitHub using the SEART GitHub Search tool [1], applying a series of filters to obtain projects with sufficient activity and CI usage. Table I summarises each filter step and the resulting project counts.

a) *PS1 – Initial Collection.*: We queried the SEART GitHub Search API for JavaScript repositories with at least 10 stars, yielding 261,739 candidates.

b) *PS2 – Activity Filter.*: We retained only projects with at least 10 commits in the study period (2024-03-01 to 2026-03-01), reducing the set to 100,811 projects. This filter ensures that the project was actively maintained during the analysis window.

c) *PS3 – GitHub Actions Adoption.*: We checked each project for the presence of at least one GitHub Actions workflow file under `.github/workflows/`. Projects without CI automation were excluded, leaving 16,226 projects.

d) *PS4 – CI Execution History.*: To confirm that GitHub Actions had actually been *executed* (not merely configured), we required at least 10 recorded workflow runs per project. This step is a novel addition over the original Dep-sCI-mitar work, which did not explicitly verify CI activity.

e) *PS5 – npm Adoption.*: Finally, we filtered for projects containing a `package.json` file at the repository root, confirming use of the npm package manager and yielding 13,991 projects.

B. Data Curation

1) *DC1 – Dependency-Update Commit Extraction.*: From the 13,991 selected projects, we extracted commits that represent *pure dependency updates*, defined as commits that modify only `package.json` and/or `package-lock.json` and change *exactly one* dependency version. Commits that also modified source files, test files, documentation, or any other file were excluded. This strict criterion ensures that each extracted commit can be attributed to a single dependency version bump, making the skip decision unambiguous.

The extraction yielded **121,453 commits** from **1,854 projects**.

2) *DC2 – Unused Dependency Detection.*: For each commit identified in DC1, we checked whether the updated dependency was actually *used* in the project source code at the time of that commit. We evaluated four detection approaches, spanning static analysis and large language model (LLM) based methods:

a) *depcheck.*: `depcheck` [2] is an AST-based static analyser that inspects all JavaScript/TypeScript files in a project and reports packages listed in `package.json` that are not imported or required anywhere. We invoked it as:

```
npx depcheck --specials=...
```

`depcheck` has been shown to achieve a precision of 94% and recall of 83% in prior evaluation [3].

b) *knip.*: `knip` [4] is a newer AST-based tool that reports unused exports, files, and dependencies in JSON format:

```
npx knip --reporter json
```

`knip` performs project-level cross-file analysis, potentially capturing indirect usage patterns that single-file tools miss.

c) *LLM-based detection (llama3.2 and qwen2.5).*: As an alternative to traditional static analysis, we explored LLM-based detection using two open-source models served locally via Ollama [5]: `llama3.2` and `qwen2.5`. For each commit, we extracted all `import/require` statements from the project source using regular expressions, then prompted the LLM to classify whether each dependency listed in `package.json` was referenced. GPU-accelerated inference was performed on an HPC cluster.

d) *Comparison rationale.*: We include multiple tools to assess how the choice of detection method affects the downstream skip analysis results. Static tools are deterministic

and widely used in practice; LLM-based tools represent an emerging alternative that may handle dynamic usage patterns or non-standard import syntax more gracefully.

3) *DC3 – CI Build Data Collection*: For each dependency-update commit identified in DC1, we retrieved its CI build outcomes via the GitHub Check Runs API:

- **Build status** of the commit itself (*success* / *failure* / *unknown*).
- **Build status** of the commit’s parent (i.e., the build result immediately before the dependency update).
- **Build duration** (in seconds) of the commit’s CI run.

After filtering commits for which CI data was available, we obtained **20,743 commits** from **1,487 projects**.

C. Skip Analysis

We applied the Dep-sClimitar skip logic [6] to each curated commit. A commit is classified as *skippable* if and only if:

$$\text{skippable} \Leftrightarrow (\text{dep} \in \text{unused} \cup \text{unused_dev}) \wedge (\text{parent_status} = \text{success}) \quad (1)$$

where *dep* is the updated dependency and *unused* / *unused_dev* are the sets of unused runtime and development dependencies reported by the detection tool. The parent-build condition ensures that we only skip builds whose pre-update state was already passing; skipping after a failing parent would risk masking an ongoing breakage.

For every skippable commit, the build duration recorded in DC3 is counted as *wasted CI time* that could have been avoided.

IV. RQ1: TODO – DEFINE RQ1 TITLE

- A. *Motivation*
- B. *Approach*
- C. *Results*
- D. *Summary*

V. RQ2: TODO – DEFINE RQ2 TITLE

- A. *Motivation*
- B. *Approach*
- C. *Results*
- D. *Summary*

VI. RQ3: TODO – DEFINE RQ3 TITLE

- A. *Motivation*
- B. *Approach*
- C. *Results*
- D. *Summary*

VII. DISCUSSION

VIII. THREATS TO VALIDITY

IX. CONCLUSION

REFERENCES

- [1] “SEART GitHub search,” <https://seart-ghs.si.usi.ch>, 2024.
- [2] “depcheck,” <https://github.com/depcheck/depcheck>, 2024.

- [3] TODO, “Todo: cite the depcheck evaluation paper (precision 94%, recall 83%),” *TODO*, TODO.
- [4] “knip,” <https://github.com/webpro/knip>, 2024.
- [5] “Ollama,” <https://ollama.com>, 2024.
- [6] S. Haiduc *et al.*, “Dep-sClimitar: Todo,” in *TODO*, TODO.